

Test Case Document

Playwright automation suite — Login, Home launcher, Patient Registration, Patient Admission, the admission dashboard, the end-to-end patient journey, and the four Nurse Station tabs: Medicine Indent, Consultancy Service, Diet Indent and Diet Dashboard. 110 cases, all automated.

1. Document control

Document	Test Case Document — PMCH Healthcare ERP Test Automation
Version	2.0
Date	05 October 2026
Prepared by	QA Automation — EMSL Dev Team
Application under test	Popular Medical College and Hospital Limited — HIS ERP (Blazor / MudBlazor)
Automation framework	Playwright Test 1.63 · TypeScript · Page Object Model
Source of truth	tests/ in this repository. Every case below maps to one test() — the spec file and line are given in each row.
Results reflected	Full regression run of 05 Oct 2026, 15:28 UTC — 99 tests in 33 m 33 s: 78 passed, 16 known fail (cases marked test.fail(), each watching a defect in the companion bug report), 2 failed and 3 skipped . The smoke and API suites were not part of that cycle and are marked NOT RUN .
Companion document	docs/BUG_REPORT.pdf — the 23 defects raised against these cases, with screenshots and video.

2. Scope and environment

In scope	Authentication, module launcher, Patient Registration list and create-patient wizard, New Admission wizard (Patient / Contact Person / Advance Payment / Admission Detail), the admission dashboard — looking an admission up and printing its paperwork — host availability, one end-to-end register-then-admit-then-indent flow, and the four Nurse Station tabs the ward works from: Medicine Indent and its Verify Indent pane, Consultancy Service (the cart, the bill and the cancel-with-reason dialog), Diet Indent, and Diet Dashboard (the date range, the search, both grids, and Stop / Continue).
Out of scope	OPD, Pharmacy, Diagnostic, HR, Accounts and Billing modules; the ID-card print and the admission dashboard's list-level <i>Print List / Print Package List</i> buttons; what is <i>inside</i> a printed document, which is checked as a valid, non-empty PDF and no further; role-based permissions beyond the Administrator account; performance and security testing.

Environment	ERP web application over HTTPS on the hospital host (BASE_URL in .env; self-signed certificate, so ignoreHTTPSErrors is on). Live database — see the warning in section 5.
Browser	Chromium (Desktop Chrome profile) at 1920×1080, headed, one worker. Firefox and WebKit projects exist in playwright.config.ts but are commented out; mobile viewports are not enabled.
Test account	Administrator account read from ERP_USERNAME / ERP_PASSWORD. Credentials never appear in source control or in this document.
Session handling	tests/auth.setup.ts logs in once and saves the browser state to playwright/.auth/user.json. Signed-in suites reuse it; signed-out suites clear it with test.use({ storageState: { cookies: [], origins: [] } }).
Timeouts	90 s per test, 15 s per assertion, 30 s per action, 45 s per navigation — raised from the Playwright defaults because the host is slow. Three cases raise their own further (noted in their rows).

3. Legend

Priority	P1 — blocks the release if it fails (login, core patient flow). P2 — important, has a workaround. P3 — cosmetic or defensive.
Status	PASS passed in the 05 Oct 2026 run · KNOWN FAIL failed <i>as designed</i> — the case is marked test.fail() and asserts the behaviour the module is supposed to have, so the run reports it as expected-to-fail and will report it as <i>unexpectedly passing</i> the day the defect is fixed; the defect it watches is named in the cell · FAIL failed unexpectedly, defect raised · SKIPPED the live ward had no data the case needed · NOT RUN not part of that cycle
Writes to live data	Rows marked WRITES leave a real record in the hospital database. Every other case is read-only.

4. Coverage summary

SUITE	SPEC FILE	CASES	PASS	KNOWN FAIL	FAIL	SKIPPED	NOT RUN	SESSION
API — Health	tests/api/health.spec.ts	2	—	—	—	—	2	None
Setup — Authentication	tests/auth.setup.ts	1	1	—	—	—	—	Creates it
Smoke — Login	tests/smoke/login.smoke.spec.ts	5	—	—	—	—	5	Signed out
Smoke — Admission	tests/smoke/admission.smoke.spec.ts	4	—	—	—	—	4	Signed in
Regression — Login	tests/regression/login.regression.spec.ts	7	7	—	—	—	—	Signed out
Regression — Home	tests/regression/home.regression.spec.ts	7	7	—	—	—	—	Signed in (007 signed out)

SUITE	SPEC FILE	CASES	PASS	KNOWN FAIL	FAIL	SKIPPED	NOT RUN	SESSION
Regression — Registration	tests/regression/registration.regression.spec.ts	6	6	—	—	—	—	Signed in
Regression — Admission	tests/regression/admission.regression.spec.ts	2	2	—	—	—	—	Signed in
Regression — Patient journey	tests/regression/admission.regression.spec.ts	1	1	—	—	—	—	Signed out
Regression — Admission dashboard	tests/regression/admission-dashboard.regression.spec.ts	7	7	—	—	—	—	Signed in
Regression — Nurse Station	tests/regression/nurse-station.regression.spec.ts	11	11	—	—	—	—	Signed in
Regression — Consultancy Service	tests/regression/consultancy-service.regression.spec.ts	23	14	9	—	—	—	Signed in
Regression — Diet Indent	tests/regression/diet-indent.regression.spec.ts	16	15	1	—	—	—	Signed in
Regression — Diet Dashboard	tests/regression/diet-dashboard.regression.spec.ts	18	7	6	2	3	—	Signed in
Total		110	78	16	2	3	11	

The 99 cases that ran are the setup plus every regression suite. **Known fail** is not a red build: all 16 are cases marked `test.fail()`, which Playwright counts as expected — the run's own figures are 94 expected, 2 unexpected, 3 skipped. The two real failures are DD-02 and DD-13, both BUG-018, and the three skips are downstream of it.

5. Data safety

The application under test is the hospital's **production** ERP. Every case marked **WRITES** leaves a real record against a real patient. They fall into three groups.

CASES	WHAT THEY LEAVE BEHIND	HOW IT IS CLEARED
TC_FLOW_001 TC_SMK_ADM_004	A registered patient, an admission holding a real bed and real advance payments, and the first medicine indent against it. TC_SMK_ADM_004 fills the wizard but stops short of the submit.	Nothing in the suite undoes them. Cancelling an admission is its own workflow under Hospital > Admission Cancel . Run TC_FLOW_001 deliberately, and expect a Masud Rana Test-<n> patient after each run.
TC_NS_004, TC_NS_005 DI-04, DI-05, DI-09, DI-12 ... DI-15 DD-14 ... DD-17	Medicine and diet indents filed against live admissions. They land as Pending , where the pharmacy and the canteen see them.	Not undone by the suite — an indent is cancelled from Nurse Station > Verify Indent . Every diet indent carries a mark unique to its case in Remarks, so they can be told from a nurse's. DD-15 ... DD-17 move diets between the two dashboard grids and <i>do</i> put the ward back as they found it, acting only on indents they filed themselves.

CASES	WHAT THEY LEAVE BEHIND	HOW IT IS CLEARED
CS-19 ... CS-22	Consultancy services charged to a live admission's bill.	Each takes its own charges back off through the tab's cancel dialog, in a finally, so a failure mid-case still cleans up — a cancellation with a stated reason, recorded in IPD Bill Change History, not a deletion. Only the rows the case added are taken off; on a long-stay patient the rest of the bill is a month of real treatment. Three of the four prefer a service the catalogue prices at zero.

Every other case is read-only: the registration and admission cases deliberately stop before any Confirm button, the negative cases on the indent forms are negative precisely because the app refuses them, and the dashboard cases only search, sort and print — a printed document changes nothing in the record it is printed from.

Both writing suites are in a describe of their own, so a run that should not write can leave them out:

```
npx playwright test tests/regression/consultancy-service.regression.spec.ts -g "CS-0|CS-1|CS-23"
npx playwright test tests/regression/diet-indent.regression.spec.ts -g "DI-0[12368]|DI-1[01]"
npx playwright test tests/regression/diet-dashboard.regression.spec.ts -g "DD-0|DD-1[0-3]"
```

6. API — Health

Spec: tests/api/health.spec.ts · Project api · no browser, no session.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_API_001 health.spec.ts:6	Health endpoint reports the host is healthy	ERP host reachable at API_URL.	1. Send GET /health with the API request context. 2. Read the status code and the response body.	GET /health	- Status is 200. - Body, trimmed, is exactly Healthy.	P1	NOT RUN
TC_API_002 health.spec.ts:13	Login page is served by the identity provider	ERP host reachable at API_URL.	1. Send GET /Account/Login. 2. Read the status code and the HTML body.	GET /Account/Login	- Status is 200. - Body contains Input.Email, proving the sign-in form rendered.	P1	NOT RUN

7. Setup — Authentication

Spec: tests/auth.setup.ts · runs before every UI project; if it fails, every signed-in case is skipped.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_SETUP_001 auth.setup.ts:10	Log in once and persist the session for the suite	.env holds valid ERP_USERNAME / ERP_PASSWORD.	1. Open /Account/Login. 2. Enter the administrator username and password; submit. 3. Wait for the module launcher to load. 4. Save the browser storage state to disk.	Administrator account from .env	- Launcher loads with the tenant name and user role. - playwright/.auth/user.json is written and reused by every signed-in project.	P1	PASS 3.1 s

8. Smoke — Login

Spec: `tests/smoke/login.smoke.spec.ts` · signed out — the suite clears the saved session so nothing is inherited.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_SMK_LOGIN_001 <code>login.smoke.spec.ts:12</code>	Login page renders every field	No session.	1. Open the login page. 2. Wait for the form to load. 3. Inspect each control.	—	- Username placeholder is <i>Enter your username</i>. - Password placeholder is <i>Enter your password</i>. - Submit button reads <i>Log in</i>. - Remember-me and Forgot-password are visible.	P1	NOT RUN
TC_SMK_LOGIN_002 <code>login.smoke.spec.ts:23</code>	Valid credentials land on the module launcher	No session; administrator account is active.	1. Open the login page. 2. Enter the administrator username and password; submit. 3. Wait for the launcher.	Administrator account from <code>.env</code>	- Launcher is displayed. - Tenant reads <i>Popular Medical College and Hospital</i>. - Role reads <i>Administrator</i>.	P1	NOT RUN
TC_SMK_LOGIN_003 <code>login.smoke.spec.ts:35</code>	Invalid password is rejected	No session.	1. Open the login page. 2. Enter a known username with a deliberately wrong password; submit.	<code>admin / wrong-password-on-purpose</code>	- Invalid-login error is shown. - User stays on the login page; no session is created.	P1	NOT RUN
TC_SMK_LOGIN_004 <code>login.smoke.spec.ts:42</code>	Unknown user is rejected	No session.	1. Open the login page. 2. Enter a username that does not exist with any password; submit.	<code>no-such-user-12345 / whatever</code>	- Invalid-login error is shown. - The message does not reveal whether the account exists.	P1	NOT RUN
TC_SMK_LOGIN_005 <code>login.smoke.spec.ts:49</code>	Unauthenticated visit redirects to login	No session.	1. Navigate straight to the application root <code>/</code> .	—	- URL becomes <code>/Account/Login</code>. - The login form is rendered.	P1	NOT RUN

9. Smoke — New Admission wizard

Spec: tests/smoke/admission.smoke.spec.ts · signed in on the saved session · page /hospital/patientadmission. Nothing here presses *Create Patient Admission*; the end-to-end submit is TC_FLOW_001.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_SMK_ADM_001 admission.smoke.spec.ts:18	Wizard opens on the Patient tab with its required fields ready	Signed in as Administrator.	1. Open the New Admission wizard. 2. Wait for the page to load. 3. Inspect the Patient tab and its fields.	—	• Patient tab is active. • UHID is editable. • Full Name, Surname, Patient Mobile are editable; DOB and Blood Group are visible.	P1	NOT RUN
TC_SMK_ADM_002 admission.smoke.spec.ts:34	The four marked tabs open in order	Signed in; wizard loaded.	1. Open Patient — check Full Name rendered. 2. Open Contact Person — check Guardian Name and Relation. 3. Open Advance Payment — check Select Card, BKash Payment, Payment (Cash). 4. Open Admission Detail — check Select Department, Admitted to, Bed/Cabin.	—	• Each tab goes active when opened. • The pane behind each tab renders its own fields — a tab that switches without its fields arriving still fails.	P1	NOT RUN
TC_SMK_ADM_003 admission.smoke.spec.ts:57	A patient loads by UHID into the Patient tab	Signed in; at least one patient in the Patient Registration grid for the selected date range.	1. Open Patient Registration and read the UHID from the first grid row. 2. Open the New Admission wizard. 3. Search that UHID.	UHID read live from the grid (10+ digits)	• Full Name, Patient Mobile and DOB are all filled from the registration record — none is empty.	P1	NOT RUN

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_SMK_ADM_004 admission.smoke.spec.ts:80	The tabs take entry in order, up to the submit WRITES	Signed in; wizard loaded. Test timeout raised to 240 s.	1. Fill the Contact Person tab; confirm Guardian Name holds what was typed. 2. Fill the Advance Payment tab (card, bKash, cash). 3. Read the page's own <i>Total</i>: panel. 4. Press Next into Admission Detail; choose Department and Admitted-to. 5. Wait for the bed list to populate, pick the first real bed. 6. Search and pick an assigned doctor.	Guardian Test– <n> · Card VISA 1000, bKash 2000, Cash 3000 · HDU / HDU · doctor search Dr	• Total panel shows 6000 — the app's own sum, which is the honest proof all three amounts registered. • Bed list loads more than the placeholder option and a bed is selected. • Assigned doctor contains the searched text. • <i>Create Patient Admission</i> is enabled — and deliberately not pressed.	P1	NOT RUN

10. Regression — Login

Spec: `tests/regression/login.regression.spec.ts` · signed out. The form relies on HTML5 required rather than a server-rendered validation summary, so the empty-field cases assert constraint validity plus "the form was never submitted" instead of hunting for an error banner.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_LOGIN_001 <code>login.regression.spec.ts:16</code>	Login page is displayed with all required elements	No session.	1. Open the login page and wait for it to load. 2. Inspect each field, the submit button and the error area.	—	• Username editable, placeholder <i>Enter your username</i>. • Password editable, <code>type="password"</code>, placeholder <i>Enter your password</i>. • Submit enabled and reads <i>Log in</i>. • Remember-me and Forgot-password visible; no error banner.	P1	PASS 3.0 s
TC_LOGIN_002 <code>login.regression.spec.ts:79</code>	Valid credentials land on the home page	No session; administrator account active.	1. Open the login page. 2. Enter the administrator username and password; submit. 3. Wait for the launcher.	Administrator account from <code>.env</code>	• Launcher loads. • Tenant reads <i>Popular Medical College and Hospital</i>; role reads <i>Administrator</i>. • No error banner.	P1	PASS 2.9 s
TC_LOGIN_003 <code>login.regression.spec.ts:32</code>	Invalid username with a valid password is rejected	No session.	1. Open the login page. 2. Enter a non-existent username with the real password; submit.	<code>no_such_user_9912</code> + real password	• Invalid-login error is shown; the user is not signed in.	P1	PASS 3.2 s
TC_LOGIN_004 <code>login.regression.spec.ts:41</code>	Valid username with an invalid password is rejected	No session.	1. Open the login page. 2. Enter the real username with a wrong password; submit.	Real username + <code>WrongPass!123</code>	• Invalid-login error is shown; the user is not signed in.	P1	PASS 3.9 s
TC_LOGIN_005 <code>login.regression.spec.ts:50</code>	Empty username is blocked by field validation	No session.	1. Open the login page. 2. Leave Username blank, fill Password; submit.	Username (<i>blank</i>) + real password	• Username reports a required-field constraint violation. • The form is never submitted — the browser stays on the login page.	P2	PASS 2.8 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_LOGIN_006 login.regression.spec.ts:60	Empty password is blocked by field validation	No session.	1. Open the login page. 2. Fill Username, leave Password blank; submit.	Real username + password (<i>blank</i>)	- Password reports a required-field constraint violation. - The form is never submitted.	P2	PASS 3.2 s
TC_LOGIN_007 login.regression.spec.ts:70	Both fields empty are blocked by field validation	No session.	1. Open the login page. 2. Submit with both fields blank.	—	- Both fields report a required-field constraint violation. - The form is never submitted.	P2	PASS 3.2 s

11. Regression — Home / module launcher

Spec: `tests/regression/home.regression.spec.ts` · signed in, except `TC_HOME_007`. The launcher is a MudBlazor shell, so module tiles are plain text nodes rather than links — they are matched exactly, because a substring match also hits the hidden language dropdown.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_HOME_001 <code>home.regression.spec.ts:19</code>	Home page is displayed with all required elements	Signed in as Administrator.	1. Open the home page and wait for it to load. 2. Inspect the header chrome.	—	- Tenant name visible and not empty. - User name, user role and language selector all visible.	P1	PASS 2.5 s
TC_HOME_002 <code>home.regression.spec.ts:30</code>	Core ERP modules are listed on the launcher	Signed in as Administrator.	1. Open the home page. 2. Look for each core module tile by exact name.	REGISTRATION, OPD, Pharmacy, Diagnostic, HR, ACCOUNT	All six tiles are visible; the failure message names the missing module.	P1	PASS 2.9 s
TC_HOME_003 <code>home.regression.spec.ts:39</code>	Tenant is the hospital the session belongs to	Signed in as Administrator.	1. Open the home page. 2. Read the tenant name in the header.	—	Tenant matches <i>Popular Medical College and Hospital</i> (case-insensitive).	P1	PASS 3.2 s
TC_HOME_004 <code>home.regression.spec.ts:46</code>	Signed-in user context is shown	Signed in as Administrator.	1. Open the home page. 2. Read the user role and user name.	—	- Role matches <i>Administrator</i>. - User name is not empty.	P2	PASS 2.4 s
TC_HOME_005 <code>home.regression.spec.ts:54</code>	A module tile that does not exist is not rendered	Signed in as Administrator.	1. Open the home page. 2. Look for a module name that cannot exist.	NO_SUCH_MODULE_9912	No such tile is rendered — negative control proving the tile matcher is not matching everything.	P3	PASS 2.2 s
TC_HOME_006 <code>home.regression.spec.ts:61</code>	Reloading keeps the user on the launcher	Signed in as Administrator.	1. Open the home page. 2. Reload (tolerant of the host's HTTP 429 cool-off — see BUG-003). 3. Wait for the launcher again.	—	- Launcher loads again; the session is not lost. - The REGISTRATION tile is still visible.	P2	PASS 2.9 s
TC_HOME_007 <code>home.regression.spec.ts:78</code>	An unauthenticated visitor is sent to the login page	Session deliberately cleared.	1. Navigate to the home page with no session.	—	- URL becomes <code>/Account/Login</code>. - The login form renders and the tenant header is hidden — no launcher content leaks.	P1	PASS 2.7 s

12. Regression — Patient Registration

Spec: `tests/regression/registration.registration.spec.ts` · signed in. Follows the module the way a user reaches it — REGISTRATION tile on the launcher, which loads that module's menu into the left drawer, then PATIENT REGISTRATION in the drawer. Nothing here submits the create-patient wizard, because confirming a registration would leave a real patient behind on every run.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_REG_001 <code>registration.registration.spec.ts:23</code>	REGISTRATION tile loads the module menu into the sidebar	Signed in; on the launcher.	1. Open the home page. 2. Confirm the drawer carries no PATIENT REGISTRATION link yet. 3. Click the REGISTRATION tile.	—	- Before the click: the link genuinely does not exist (count 0). - After: the REGISTRATION group and the PATIENT REGISTRATION link appear, the link pointing at <code>/hospital/newregistration</code>.	P1	PASS 4.5 s
TC_REG_002 <code>registration.registration.spec.ts:44</code>	PATIENT REGISTRATION opens the patient list	Signed in; on the launcher.	1. Click the REGISTRATION tile. 2. Click PATIENT REGISTRATION in the drawer.	—	- The patient list page loads. - URL ends with <code>/hospital/newregistration</code>.	P1	PASS 4.3 s
TC_REG_003 <code>registration.registration.spec.ts:57</code>	Patient list is displayed with all required elements	Signed in; patient list open.	1. Open the patient list. 2. Inspect the filter bar and toolbar.	—	- Show From and Show To date filters visible; Show button enabled. - Add New visible and pointing at <code>/patients/new</code>. - Search box editable.	P1	PASS 2.7 s
TC_REG_004 <code>registration.registration.spec.ts:71</code>	The grid lists patients under the expected columns	Signed in. At least one patient registered within the grid's default date range (today) — see BUG-001.	1. Open the patient list. 2. Read the header row. 3. Wait for the first data row.	Live hospital data	- Exactly six headers, in order: NAME, UHID, REG DATE, PHONE NUMBER, DOB, ACTION. - At least one data row is visible.	P1	PASS 10.3 s
TC_REG_008 <code>registration.registration.spec.ts:89</code>	Add New opens the create-patient form	Signed in; patient list open.	1. Open the patient list. 2. Click Add New.	—	- The create-patient wizard loads. - URL ends with <code>/hospital/patients/new</code>.	P1	PASS 4.3 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_REG_009 registration.regression.spec.ts:102	The create-patient form opens on the Patient tab	Signed in; create-patient wizard open.	1. Open the create-patient wizard directly. 2. Read the aria-selected state of each tab.	—	- Patient tab is selected. - Contact Person and Corporate Client are not selected.	P2	PASS 3.3 s

ID gap. TC_REG_005, TC_REG_006 and TC_REG_007 — the grid search and row-action cases — were removed from the spec and are not currently automated. They are listed as a coverage gap in section 15; the surviving IDs were left unrenumbered so that earlier runs and defect reports still line up.

13. Regression — New Admission

Spec: `tests/regression/admission.regression.spec.ts` · signed in. The wizard admits a patient who is already registered, so the UHID is read out of the registration grid rather than hard-coded — the grid holds live hospital data and no patient in it is guaranteed to still be there next week.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_ADM_001 <code>admission.regression.spec.ts:48</code>	New Admission opens from the Hospital module menu	Signed in; on the launcher.	1. Open the home page. 2. Open the Hospital module and find the New Admission link. 3. Click it.	Route <code>/hospital/patientadmission</code>	• The admission wizard loads. • UHID is editable and the Patient tab is active.	P1	PASS 5.0 s
TC_ADM_002 <code>admission.regression.spec.ts:63</code>	Searching a UHID loads that patient into the wizard	Signed in. At least one patient registered in the last year — the list opens filtered to today (BUG-001), so the case widens the window itself.	1. Open Patient Registration and widen <i>Show From</i> to the last year. 2. Read the name and the UHID off the first grid row, in one go, so a re-render cannot hand back two different patients. 3. Open the New Admission wizard. 4. Search that UHID.	UHID and name read live from the grid	• Full Name in the wizard equals the name in the grid, normalised — the grid cell wraps the name and the wizard's field does not. • Patient Mobile and DOB are filled from the registration record.	P1	PASS 13.2 s — was the 03 Oct failure; AUT-001 is fixed, BUG-001 stands

14. Regression — Patient journey (end to end)

Spec: `tests/regression/admission.regression.spec.ts` · signed out, because logging in is the first thing it checks. One patient, followed the way the hospital actually takes one on. It is one test and not three because the admission wizard needs a patient who is registered and not already on a ward, and the only way to be sure of that is to have just registered them. Retries are disabled: a retry would register a second patient rather than tell us anything new.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_FLOW_001 admission.regression.spec.ts:112	Log in, register a patient, then admit them WRITES	No session. A free bed in the target ward. Test timeout raised to 600 s; retries disabled.	1. Log in with the administrator account and reach the launcher. 2. Register the patient: open the create-patient wizard, fill the Patient tab, confirm the DOB picker wrote 4/10/1998, press Confirm Registration and capture the new Patient ID. 3. Admit them: open the New Admission wizard and search the UHID just issued. 4. Fill Contact Person, then Advance Payment; read the page's own <i>Total:</i> panel. 5. Press Next into Admission Detail; fill department, ward, bed and assigned doctor. 6. Press Create Patient Admission and capture the confirmation text. 7. Verify on the ward: open Hospital > Dashboard (the admission dashboard) and search the patient name as registered; read the admission number off the row. 8. Find it by number and print: search that admission number, then press every print button in the row's Action cell, one after another, and read back the PDF each one produced.	Patient Masud Rana Test-<n>, Male, DOB 10 Apr 1998, A+, mobile 01876765454, NID 98876756757, Tangail / Bhuapur · Card VISA 1000, bKash 2000, Cash 3000 · HDU / HDU	• Login succeeds and the launcher loads. • Registration returns a Patient ID of 10 or more digits. • The wizard pulls up that exact patient — name and mobile match what was registered. • Total panel shows 6000. • The confirmation carries no <i>error / failed / invalid</i> wording. • An admission row exists on the admission dashboard: admission number of 10+ digits, carrying the patient's mobile and a bed in the ward admitted to. • Searching that admission number returns exactly one row, and it is this patient. • At least four print buttons, each producing a PDF (%PDF- header, over 1 KB).	P1	PASS 1 m 48 s

Deliberately not asserted: that the advance payments reach the admission. On the runs so far they did not — Hospital > Advance Dashboard carried no row for the admission this flow created, even though the wizard summed the three amounts to 6000 before the submit. The case asserts only what it can stand behind, and the gap is raised separately as **BUG-002**.

15. Regression — Admission dashboard

Spec: `tests/regression/admission-dashboard.regression.spec.ts` · signed in. Route `/hospital/patientlist-dashboard` — Hospital > **Dashboard**, the list of everyone currently admitted. This is where an admission is looked up after the fact, so the cases are the two things the page is used for: finding one admission in it, and printing that admission's paperwork. The grid holds live hospital data, so no case hard-codes an admission — each reads one out of the first row and searches for what it just read. Read-only throughout.

One trap, recorded here because it is not a defect: the NAME column prints the patient's title in front of the name (Mr Jalal Miah) but the search box matches the name as it was registered, without it. Searching a name copied straight out of the grid returns nothing; Jalal Miah returns the row. TC_ADMD_004 strips the title before searching.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_ADMD_001 <code>admission-dashboard.regression.spec.ts:40</code>	Dashboard opens from the Hospital module menu	Signed in; on the launcher.	1. Open the home page. 2. Open the Hospital module and find the Dashboard link in the drawer. 3. Click it.	Route <code>/hospital/patientlist-dashboard</code> — the link is reached by href, because it is labelled just "Dashboard"	- The admission dashboard loads with rows in the grid. - Search is editable; <i>Print List</i> and <i>Print Package List</i> are on screen.	P1	PASS 5.4 s
TC_ADMD_002 <code>admission-dashboard.regression.spec.ts:59</code>	The grid lists admissions under the expected columns	Signed in. At least one patient admitted.	1. Open the admission dashboard. 2. Read the grid's header row. 3. Read the ADM.NO and CABINNO cell of every row on the page.	Eleven columns: ADM.NO, ADM.DATE, ADM.TIME, NAME, CABINNO, ASSIGNED DOCTOR, PHONE, ADDRESS, CABINTYPE, PACKAGE, ACTION	- Exactly those eleven headers, in that order. - No row is missing its admission number or its bed — the two every admitted patient has.	P2	PASS 3.5 s
TC_ADMD_003 <code>admission-dashboard.regression.spec.ts:79</code>	Searching an admission number finds that admission	Signed in. At least one patient admitted.	1. Open the admission dashboard and read the first row. 2. Type its admission number into Search.	Admission number read live from the grid	- Exactly one row comes back — an admission number is unique. - Its admission number and patient name are the ones read in step 1.	P1	PASS 4.1 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_ADMD_004 admission-dashboard.regression.spec.ts:93	Searching a patient name finds their admission	Signed in. At least one patient admitted.	1. Open the admission dashboard and read the first row. 2. Drop the title from the name (Mr Jalal Miah → Jalal Miah). 3. Type the remainder into Search.	Patient name read live from the grid	• The admission read in step 1 is among the rows returned. • Its name matches. <i>Not</i> asserted as a single row: a ward holding two patients of one name is ordinary.	P1	PASS 4.1 s
TC_ADMD_005 admission-dashboard.regression.spec.ts:111	Searching a phone number finds that admission	Signed in. Skipped if the admission at the top of the grid carries no phone number.	1. Open the admission dashboard and read the first row. 2. Type its phone number into Search.	Phone number read live from the grid	• The admission read in step 1 is among the rows returned. • Its PHONE cell is the number searched. Not a single-row assertion — a family shares a number.	P2	PASS 3.7 s
TC_ADMD_006 admission-dashboard.regression.spec.ts:125	Searching a cabin number finds the patient in that bed	Signed in. At least one patient admitted.	1. Open the admission dashboard and read the first row. 2. Type its cabin number into Search.	Cabin number read live from the grid, e.g. HDU-48	• The admission read in step 1 is among the rows returned. • Every row returned is for that bed — a bed holds one patient at a time.	P2	PASS 4.4 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_ADMD_007 admission-dashboard.regression.spec.ts:143	Every Action print button renders a PDF	Signed in. At least one patient admitted. Test timeout raised to 240 s — four documents, the largest over a megabyte.	1. Open the admission dashboard and search the first row's admission number. 2. Press the first print button in the Action cell; wait for the tab it opens. 3. Read the document back out of the browser and close the tab. 4. Repeat for every remaining print button, one at a time, left to right.	One admission, read live from the grid. Measured on 03 Oct 2026: 1 388 783 / 643 337 / 37 660 / 643 270 bytes	• At least four print buttons, each opening its own tab. • Each tab carries a real PDF — %PDF-header, over 1 KB. • Not all four are the same file: these are different documents, and four identical ones would mean the buttons share a handler.	P1	PASS 10.2 s

How a printed document is checked. The app does not navigate to a document or download one — it builds the file in the browser and opens a tab on a `blob:` URL, which renders in the browser's own PDF viewer and has no readable page behind it. The blob belongs to the dashboard, which created it, so the test fetches that URL from the dashboard and gets the actual bytes. That is what makes "the button printed something" checkable rather than just "a tab opened" — a page that failed to render would open a tab too.

Not clicked: some rows carry a cloud-arrow-down button among the print buttons. It produced nothing observable on any row tried — no tab, no download, nothing on screen — so the print run matches on the printer icon and steps over it rather than hanging on a document that never comes. Worth a question to the module owners; it is not raised as a defect because what it is meant to do is unknown.

16. Regression — Nurse Station (Medicine Indent)

Spec: `tests/regression/nurse-station.regression.spec.ts` · signed in. Route `/hospital/nurse-station`, which opens on **Medicine Indent > Add Indent** — where the ward nurse raises a medicine indent against an admitted patient: find the patient by admission number, set a priority, list the drugs and quantities, save. The cases are that flow end to end (TC_NS_004, confirmed on Verify Indent by TC_NS_005), the five ways the form is supposed to refuse, and the Verify Indent tab the pharmacy works from (TC_NS_011).

No case hard-codes a patient. Which beds are occupied changes by the hour, so each case reads a patient out of the live ward list and works with whoever that turns out to be. **TC_NS_004 and TC_NS_005 write** — each files a real indent that reaches the pharmacy as Pending; see section 5.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_NS_001 <code>nurse-station.regression.spec.ts:42</code>	Nurse Station opens from the Hospital module menu	Signed in; on the launcher.	1. Open the home page and the Hospital module drawer. 2. Find the Nurse Station link by href and click it.	Route <code>/hospital/nurse-station</code>	- The page loads with MEDICINE INDENT and ADD INDENT both active — the indent form is on screen with nothing clicked. - Save is visible.	P1	PASS 4.9 s
TC_NS_002 <code>nurse-station.regression.spec.ts:61</code>	The Add Indent form offers every control the indent needs	Signed in; Nurse Station open.	1. Assert every tab in the strip is present. 2. Open the Priority dropdown and count each priority the ward can raise under. 3. Assert Outlet, OT Room, the Product Code lookup and the Quantity box. 4. Read the item grid's headers one by one.	Six priorities from <code>NurseStationPage.PRIORITIES</code> ; item columns from <code>ITEM_COLUMNS</code>	- Every tab, every priority, and every named item column is present, in order. - Quantity is editable. A missing priority would silently narrow what the nurse can file.	P2	PASS 3.4 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_NS_003 <code>nurse-station.regression.spec.ts:98</code>	Searching an admission number opens that patient on the indent form	Signed in; at least one patient on the ward.	1. Read the admission number off the first ward card. 2. Type it into the ward search. 3. Click the card that comes back and read the patient panel.	Admission number read live off the ward list	• The search returns exactly one card — an admission number belongs to one admission. • <i>Indent for</i> carries the patient's own UHID, not the admission number searched for. • Name, Cabin and Admitted DateTime are all filled from the admission record.	P1	PASS 4.4 s
TC_NS_004 <code>nurse-station.regression.spec.ts:118</code>	A ten-line medicine indent saves against an admitted patient WRITES	Signed in; a patient on the ward. Test timeout raised to 420 s — ten lookups, ten picks, ten server round trips.	1. Select the first ward patient; choose a priority. 2. Add ten medicines through the Product Code lookup, each at its own quantity. 3. Read the item table back and compare it against what was ordered. 4. Press Save and read the confirmation.	Ten lines from <code>medicineIndent(10)</code> — one search term, a different drug per line, random quantities per run	• Priority takes: the dropdown turns <i>is-valid</i>. • Ten distinct medicines are listed, each in the quantity typed, in the order added. • Save answers <i>successful save</i>. • The form empties and Priority falls back to <i>is-invalid</i> — a table still holding ten lines would mean the confirmation was shown over an indent that never left.	P1	PASS 1 m 38 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_NS_005 <code>nurse-station.regression.spec.ts:165</code>	A saved indent appears on Verify Indent as Pending WRITES	Signed in; a patient on the ward. Test timeout raised to 240 s.	1. Read the last indent number issued before this case files its own. 2. File a two-line indent against a ward patient. 3. Open Verify Indent and wait for an indent numbered above the baseline.	Two lines from <code>medicineIndent(2)</code> . Indent numbers run in sequence, which is the only handle there is — the save confirms itself with a snackbar and no number	• The indent reaches the pharmacy under the bed it was raised for, at the priority it was raised at. • Status is Pending — raised, not yet dispensed. • It carries a user name, and a number above the baseline.	P1	PASS 36.5 s
TC_NS_006 <code>nurse-station.regression.spec.ts:198</code>	An admission number nobody is admitted under returns no patient	Signed in; Nurse Station open.	1. Search an admission number the ward could have had and does not. 2. Assert the ward list and the patient panel. 3. Clear the search and assert the list comes back.	9999999999 — shaped like a real admission number, so the search is tested rather than the input validation	• No cards come back, and the patient panel stays empty rather than keeping whoever was last on it. • Clearing the box restores the list — a filter, not a failed fetch.	P2	PASS 5.0 s
TC_NS_007 <code>nurse-station.regression.spec.ts:218</code>	Save with no patient and no priority is refused	Signed in; the form untouched.	1. Press Save on a form with nothing on it.	—	• The form answers in the markup rather than in a message: Priority is left marked <code>is-invalid</code>. • No patient, no item rows. An indent saved with no patient would be an indent nobody can serve.	P1	PASS 4.1 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_NS_008 <code>nurse-station.regression.spec.ts:234</code>	Save with a patient but no medicines is refused	Signed in; a patient on the ward.	1. Select a ward patient and choose Routine. 2. Press Save with an empty item table.	Priority Routine	• Refused with <i>please fill medicine table</i>. • Refused, not reset — the patient and the priority survive, so the nurse adds the drugs and saves rather than starting over.	P1	PASS 8.6 s
TC_NS_009 <code>nurse-station.regression.spec.ts:250</code>	A medicine with a quantity of zero is not added to the indent	Signed in; a patient on the ward.	1. Select a patient, choose Routine, pick a medicine. 2. Commit the line at a quantity of 0. 3. Commit the same medicine again at a real quantity.	Quantity 0, then a real quantity from <code>medicineIndent(1)</code>	• The zero line is not committed — a zero that got through would reach the pharmacy as an order for nothing. • The medicine is left on the form, and goes in on the second attempt — which is what says it was refused for the zero rather than for the drug.	P2	PASS 14.7 s
TC_NS_010 <code>nurse-station.regression.spec.ts:271</code>	The patient panel cannot be filled in by hand	Signed in; Nurse Station open.	1. Assert <i>Indent for</i>, Name, Cabin and Admitted DateTime are disabled. 2. Assert Item Description is disabled too.	—	• All five are disabled in the markup. The indent is bound to an admission, and a ward card is the only thing that binds it. • A typed-in name would file the indent against a patient the ward has no record of.	P2	PASS 3.5 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
TC_NS_011 nurse-station.regression.spec.ts:292	Verify Indent lists the ward's pending indents	Signed in; Nurse Station open. Skips if the ward has no indent pending.	1. Open the Verify Indent pane — its tabs and grid are not in the DOM while Add Indent is showing. 2. Assert the four status tabs and which one the pane opens on. 3. Read the grid's headers, then its rows.	Statuses Pending, Partially Served, Served, Cancelled; columns from VERIFY_COLUMNS	• All four status tabs are present and the pane opens on Pending. • Every named column is present, in order. • Assertions are made about the rows on screen, not the ward's whole backlog — the grid lists sixteen indents and stops, ordered by priority rather than by time.	P2	PASS 8.3 s

17. Regression — Consultancy Service

Spec: `tests/regression/consultancy-service.regression.spec.ts` · signed in. Route `/hospital/nurse-station`, the **Consultancy Service** tab — where the ward puts a consultant's visit, or any other chargeable service, onto an admitted patient's bill.

How the tab works, since nothing on it says. The ward list fills a patient panel of four read-only boxes. Under that is one line entry row — Service, Doctor Name, Service Date, Schedule, Rate, Qty — and *Enter on Qty* is what commits a line: there is no Add button, the only button between the two grids being Save. The upper grid is the **cart**, lines costed but not yet billed, with Total Amount and a discount box under it. The lower grid is the **bill**, every consultancy already saved against the admission, with a bin that opens a cancel-with-reason dialog.

Nine cases carry KNOWN FAIL. Each one asserts the behaviour the tab is supposed to have and is marked `test.fail()`, so a run reports it as expected-to-fail — and reports it as *unexpectedly passing* on the day the module is corrected, which is the signal to drop the marker and keep the assertion. Nothing is rewritten to match the defect, which would turn it into a documented feature and leave nothing watching for the fix. The nine are raised as **BUG-009** ... **BUG-016** in the companion bug report.

What writes. Only CS-19 ... CS-22 save. Each charges a live admission and takes its own charges back off again through the tab's cancel dialog, in a `finally`, so a failure mid-case still cleans up — a cancellation with a stated reason, recorded in IPD Bill Change History, not a deletion. Each prefers a service the catalogue prices at zero, so a run that dies between the save and the cleanup has still moved nobody's bill. The cleanup takes off *only* the rows its own case added: on a long-stay patient the bill is fifteen real consultancies, and a cleanup that emptied it would unpick a month of somebody's treatment.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-01 <code>consultancy-service.regression.spec.ts:100</code>	Consultancy Service opens from the Nurse Station tab strip, with both grids	Signed in; on the launcher.	1. Open the Hospital module and click Nurse Station. 2. Assert the page lands on Medicine Indent — Consultancy Service is four tabs along. 3. Open Consultancy Service and read the entry row and both grid headers.	Columns from <code>CART_COLUMNS</code> and <code>BILL_COLUMNS</code>	- Service, Doctor Name and Schedule lookups, Service Date, Rate and Qty are all on screen; Save is enabled. - Cart and bill each carry their own named columns, in order, plus a last one for the bin.	P1	PASS 7.8 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-02 <code>consultancy-service.regression.spec.ts:136</code>	The ward list narrows to a bed or an admission number	Signed in; patients on the ward.	- Count the whole ward list. - Search a live admission number; assert one card comes back and it is that admission. - Search the ward prefix off that bed; assert every card returned matches it. - Search a term the ward does not hold.	Admission number and bed read live off the list; <code>noSuchTerm</code> from <code>consultancyService()</code>	- An admission number is unique, so it brings back exactly one card. - A bed matches by substring — "ICU" reaches NICU-11 and Neuro ICU-13 — so every card returned contains the term rather than the count being fixed. - A term no bed matches empties the list rather than falling back to all of it.	P2	PASS 12.8 s
CS-03 <code>consultancy-service.regression.spec.ts:181</code>	Picking a ward patient fills the patient panel, and none of it can be typed into	Signed in; a patient on the ward.	- Pick a ward patient and read the panel back. - Assert all four patient fields are disabled.	Ward patient at the run's rotating ward offset	<i>Inv.No</i> is the admission number off the card that was clicked — the tab names it <i>Inv.No</i>, but it is not an invoice number of its own. - Name is filled and Cabin No holds the card's bed. - All four are disabled: the ward list is the only way a patient gets onto this form. <i>Not asserted</i>: that Assigned Doctor is filled. An admission can genuinely be under no consultant yet.	P1	PASS 7.7 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-04 <code>consultancy-service.regression.spec.ts:215</code>	The Service lookup answers a search with the services it matches, and their rates	Signed in; a patient picked.	1. Search the Service lookup and read every row it offers. 2. Check each name against the term and each rate for readability. 3. Record any names the catalogue offers at more than one rate.	<code>serviceSearch</code> — one term matching a page of services	• Every service returned contains the search term; every rate reads as a number and is not below zero. • <i>Recorded, not asserted:</i> the catalogue holds entries that share a name and differ only in rate — two called "ICU Visit", at 1000 and at 0. That is the ward's data, and the lookup does show the rate beside each.	P2	PASS 11.1 s
CS-05 <code>consultancy-service.regression.spec.ts:261</code>	Picking a service puts its catalogue rate on the form, and Schedule offers two	Signed in; a patient picked. Skips if nothing matching the term carries a rate today.	1. Assert nothing is picked on arrival and the Rate box reads 0. 2. Pick the first priced service in the lookup; read the Rate box. 3. Pick a doctor through the Doctor Name lookup. 4. Open Schedule and read every row.	<code>serviceSearch</code> , <code>doctorSearch</code> ; schedules from SCHEDULES	• The service picked is the one that lands on the form, and the Rate box takes the catalogue rate. • The Doctor lookup returns the consultant searched for. • Schedule holds exactly On Time and Off Time — no placeholder, no third option.	P1	PASS 16.4 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-06 <code>consultancy-service.regression.spec.ts:297</code>	A line with no service, or no doctor, is refused	Signed in; a patient picked. Test timeout raised to 180 s — each refusal costs the full wait for a line that never arrives.	1. Press Enter on Qty with nothing picked at all. 2. Pick a service, no doctor, and press Enter on Qty.	Qty 1 both times; expected text <code>MESSAGES.missingFields</code>	• Both are refused with <i>Please select fields.</i> — a quantity on its own is not a line. • The cart stays empty. This is the guard CS-17 then walks straight through with the lookup's blank row.	P1	PASS 39.0 s
CS-07 <code>consultancy-service.regression.spec.ts:328</code>	A quantity of zero, or a negative one, is refused	Signed in; a patient picked. Test timeout raised to 180 s.	1. Fill a complete line and commit it at a quantity of 0. 2. Do the same at -5. 3. Read the service and doctor boxes back after the refusals.	Quantities 0 and -5; expected text <code>MESSAGES.badQuantity</code>	• Both are refused and named as quantity problems; neither reaches the cart. • The refusal leaves the service and the doctor where they were, so the nurse corrects the number rather than picking the service again.	P1	PASS 1 m 32 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-08 <code>consultancy-service.regression.spec.ts:364</code>	Enter on Qty adds the line at rate × qty, and Total Amount is the cart	Signed in; a patient picked.	- For each quantity in the run's set: pick the next service in the lookup, add a doctor and a schedule, press Enter on Qty. - Read each cart line back against the catalogue entry it came from. - Sum the cart and compare against Total Amount. - Read the entry row after the last commit.	Quantities from <code>consultancyService()</code> , schedules alternating On Time / Off Time	- Each line carries the service picked, the quantity typed, the catalogue rate, and an amount of rate × qty. - The schedule reaches the line. - Total Amount equals the sum of the cart. - The entry row clears Qty and Doctor but keeps the service and its rate — the ward often bills the same service twice over.	P1	PASS 1 m 4 s
CS-09 <code>consultancy-service.regression.spec.ts:410</code>	A cart line can be taken off again before it is saved	Signed in; a patient picked; two services in the catalogue to pick from.	- Commit two different services at qty 1 each. - Assert Total Amount is the two amounts added. - Press the bin on the first row.	Two services off one lookup search	- The bin acts on the first click — no confirmation, no message; the row simply goes. - Exactly one row goes, and it is the right one. - Total Amount drops to the surviving line's amount.	P2	PASS 1 m 3 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-10 <code>consultancy-service.regression.spec.ts:450</code>	Save with an empty cart is refused	Signed in; a patient picked; cart empty.	- Count the bill rows. - Press Save with nothing in the cart.	Expected text <code>MESSAGES.emptyCart</code>	- The save is refused and says so. - The bill is unchanged — an empty Save puts nothing on it.	P1	PASS 9.9 s
CS-11 <code>consultancy-service.regression.spec.ts:471</code>	The bill lists what the ward has already charged this admission	Signed in. Walks the first ten beds for a patient with a bill; skips if none has one. Test timeout raised to 180 s.	- Step through the ward until a patient with bill rows turns up. - Read every row of the bill and check it against itself.	Live bill data — a patient with nothing charged yet is the normal case on a quiet ward	- Every charge names a service, is billed in a quantity above zero at a rate not below zero, and carries amount = rate × qty – discount. - Every charge names who served it and carries a date in <code>d/M/yyyy h:mm:ss AM</code> form. <i>Recorded, not asserted:</i> the services on the bill with no doctor against them — CS-17 is how they get there.	P1	PASS 7.8 s
CS-12 <code>consultancy-service.regression.spec.ts:515</code>	The cart is emptied when another patient is picked	Signed in; two different patients on the ward. Test timeout raised to 180 s.	- Pick the patient in bed A and commit one service to the cart. - Click the patient in bed B — the mis-click this case is about. - Read the patient panel, the cart and Total Amount.	Two consecutive ward patients; one service at qty 1	Expected: picking a patient starts that patient's bill — the cart and Total Amount are cleared. Actual: the panel moves to patient B while A's services stay in the cart and A's money stays in Total Amount. Save from there bills B for A's consultants, and nothing on screen says so. Raised as BUG-009.	P1	KNOWN FAIL 54.3 s · BUG-009

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-13 <code>consultancy-service.regression.spec.ts:568</code>	The cart is emptied when the tab is left and come back to	Signed in; a patient picked. Test timeout raised to 180 s.	1. Commit one service to the cart. 2. Go to Medicine Indent and back to Consultancy Service. 3. Read the cart once the pane has settled, then the patient panel.	One service at qty 1	• Expected: a cart with no patient against it should not exist. • Actual: the round trip clears the patient panel and leaves the cart holding lines, with a figure in Total Amount, against nobody at all. Raised as BUG-010. • Read after the pane settles, not the moment it paints: coming back repaints the grid empty for about a second before refilling it.	P1	KNOWN FAIL 41.3 s - BUG-010
CS-14 <code>consultancy-service.regression.spec.ts:620</code>	The catalogue rate cannot be typed over on the form	Signed in; a patient picked. Skips if nothing matching the term carries a rate today.	1. Pick a priced service and read its catalogue rate. 2. Type the Rate box down to 1 and commit the line. 3. Read the rate and amount the cart line carries.	Rate typed <i>down</i> to 1, never up — the cart is never saved here, and a case that typed a rate up and then died before clearing it would leave a bigger number on screen than the ward meant to charge	• Expected: the Rate box is disabled; the line goes in at the catalogue rate. A ward nurse adding a consultant's visit is not the person who prices it. • Actual: the box is editable and the line carries whatever was typed. A service the catalogue prices at 1000 goes in at 1. Raised as BUG-011; CS-22 is the same finding followed through the save.	P1	KNOWN FAIL 52.9 s - BUG-011

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-15 <code>consultancy-service.regression.spec.ts:663</code>	The Service Date chosen survives committing the line	Signed in; a patient picked.	1. Read the date the box opened on — read back rather than compared against this machine's clock, which disagrees with the app either side of midnight. 2. Set the box to a back-date; pick a service, a doctor and a schedule. 3. Assert the back-date is still there right up to the Enter. 4. Commit the line and read the box once it has settled.	backdated from <code>consultancyService()</code>	• Expected: the date chosen survives the commit, so a service performed three days ago can be entered as three days ago. • Actual: the box takes the date and keeps it through the whole of the entry, then drops it at the Enter — a render after the row lands — back to the day the tab opened on. The saved row then carries the moment of the save rather than either. Raised as BUG-012.	P2	KNOWN FAIL 21.5 s · BUG-012

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-16 <code>consultancy-service.regression.spec.ts:720</code>	Service Change comes off the amount	Signed in; a patient picked. Skips if nothing matching the term costs more than the discount.	1. Pick a service priced above the discount and fill the line. 2. Type the discount into the Service Change box under Total Amount. 3. Commit the line and read it back.	discount from <code>consultancyService()</code>	• Expected: the discount reaches the cart line and comes off the amount. • Actual: the box takes a number and keeps it, so nothing on screen suggests it was ignored — but neither the cart line nor the saved row carries it. The form has the box, the cart a column for it and the bill a Discount column, and nothing reads any of it. Raised as BUG-013.	P2	KNOWN FAIL 19.1 s · BUG-013
CS-17 <code>consultancy-service.regression.spec.ts:769</code>	The blank row in the Doctor Name lookup cannot stand in for a doctor	Signed in; a patient picked. Skips if the doctor list no longer opens on a blank row. Test timeout raised to 180 s.	1. Open the unsearched Doctor Name lookup and confirm the first row carries no code and no name. 2. Pick a service, then click that blank row. 3. Commit the line.	The lookup's own first row — a blank record in the consultant list	• Expected: Doctor Name is required, and a blank record is not a doctor; the commit is refused with <i>Please select fields</i>. the way CS-06 is. • Actual: clicking the blank row satisfies the guard. The line is committed, and saves, with no doctor against it — which is how the blanks CS-11 records get onto the bill. Raised as BUG-014.	P1	KNOWN FAIL 47.9 s · BUG-014

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-18 <code>consultancy-service.regression.spec.ts:813</code>	A lookup search that matches nothing says so	Signed in; a patient picked.	1. Search the Service lookup for a term no service matches. 2. Assert nothing selectable comes back. 3. Look for an empty-state message in the panel.	noSuchTerm	• Nothing selectable comes back, which is right. • Expected: the panel says there is no such service. • Actual: an empty bordered row with no text in it — the same picture as a lookup that has not finished loading. Raised as BUG-015.	P3	KNOWN FAIL 27.9 s - BUG-015
CS-23 <code>consultancy-service.regression.spec.ts:844</code>	The bill lists every service charged, however many there are	Signed in. Walks the first eighteen beds for a patient already at the cap; skips if none is. Test timeout raised to 180 s.	1. Step through the ward until a patient's bill is at fifteen rows. 2. Search the tab for a pager, a page-size box or a total. 3. Count the rows the bill grid lists.	BILL_ROW_CAP = 15	• Expected: the bill lists every service charged, or says it is not. • Actual: the grid stops at fifteen rows, and there is no pager, no page-size box and no total anywhere on the tab. A sixteenth consultancy is charged and then cannot be seen here at all — and because the bin lives on the row, it cannot be cancelled from this tab either. Raised as BUG-016. • This bit the suite before it was understood, which is why every writing case goes through <code>patientBelowBillCap</code>.	P1	KNOWN FAIL 13.8 s - BUG-016

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-19 consultancy-service.regression.spec.ts:909	Save puts the cart on the bill and clears it WRITES	Signed in; a patient with room below the bill cap; a free service in the catalogue. Test timeout raised to 240 s.	1. Baseline the bill, then commit one free service to the cart. 2. Press Save. 3. Read the cart, the bill count and the new row. 4. Cleanup: cancel off only the rows the baseline cannot account for.	A service the catalogue prices at zero, qty 1	• The save is acknowledged; the cart is emptied — the one place the tab does clear it. • The bill grows by exactly one row, appended last, carrying the same service, quantity, rate and amount as the cart line, and naming who charged it. • Everything that was on the bill before is still on it, unchanged.	P1	PASS 46.4 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-20 consultancy-service.regression.spec.ts:970	A saved service is cancelled with a reason, and the rest of the bill is left alone WRITES	Signed in; a patient with room for two below the cap; a free service. Test timeout raised to 240 s.	1. Save two lines of the same free service at different quantities, so the two can be told apart — a Save stamps every row it writes with the same moment, so ServiceDate separates nothing. 2. Open the cancel dialog on the first of them; read what it names and which reason chips it offers. 3. Press <i>Keep service</i> and assert nothing was cancelled. 4. Cancel it properly, with a reason. 5. Cleanup: cancel off what is left of this case's own rows.	Two lines at qty 1 and qty 2; reasons from CANCEL_REASONS	• The dialog names the service it is about to take off and offers every reason chip. • <i>Keep service</i> leaves the bill at two. • The cancellation is acknowledged, takes off exactly one row, and it is the right one — the other survives. • The rest of the bill is untouched.	P1	PASS 1 m 41 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
CS-21 <code>consultancy-service.regression.spec.ts:1046</code>	The cancel dialog refuses a blank reason WRITES	Signed in; a patient with room below the cap; a free service. Test timeout raised to 240 s.	1. Save one free service, then open the cancel dialog on it. 2. Confirm with the reason box empty. 3. Fill a single character and confirm again. 4. Cleanup: close the dialog, then cancel off this case's own row.	Reasons ' ' and x	• A blank reason is refused inline — a message under the box, not a snackbar — and the dialog stays open with the service still on the bill behind it. • A one-character reason is refused the same way. • The bill count never moves.	P1	PASS 50.9 s
CS-22 <code>consultancy-service.regression.spec.ts:1108</code>	A rate typed over the catalogue rate does not reach the bill WRITES	Signed in; a patient with room below the cap; a <i>priced</i> service, because the whole of this case is about what happens to the price. Test timeout raised to 240 s.	1. Pick a priced service and read its catalogue rate. 2. Type the Rate box down to 1, commit, and Save. 3. Read the rate on the saved bill row. 4. Cleanup: cancel off this case's own row.	Rate typed down to 1, never up — this is a live bill, and the finding is the same either way round	• Expected: the bill carries the catalogue rate whatever was typed at the nurse station. • Actual: the rate a ward nurse types is the rate the patient is charged. CS-14 is the same finding at the form; this is it on the bill, which is where it matters. Raised as BUG-011.	P1	KNOWN FAIL 46.0 s · BUG-011

18. Regression — Diet Indent

Spec: `tests/regression/diet-indent.regression.spec.ts` · signed in. Route `/hospital/nurse-station`, the **Diet Indent** tab — where the ward nurse orders meals for an admitted patient: pick the patient off the ward list, set an outlet and a priority, list the food patterns, save. Medicine Indent, the tab next to it, is section 16; the two share a route and almost no markup, which is why they have a page object each.

Three things worth knowing before reading the cases. (1) A patient cannot be given the same food pattern twice while the first order is still pending — the line is turned away at Enter with *Already has indent for this food pattern*. That is the duplicate guard this module has, and DI-14 is the case about it. It is also why no case names a food pattern: each asks for the first one the ward will still take, because every save leaves the next case one pattern less to choose from. (2) **The quantity is not recorded.** Every line lands in the indent table as 1, whether 5, 9 or 0 is typed, and reaches Verify Indent as 1 as well — DI-06 is the case that says so, and it is why every other case reads quantities back rather than assuming them. (3) Nothing is answered inline: every refusal and every save arrives as a Blazorise snackbar, in the same element with the same classes, so the text is the only thing separating them.

DI-04, DI-05, DI-09 and DI-12 ... DI-15 write. Each files at least one real diet indent against a real patient on a live ward; DI-15 files two. They land as Pending on Verify Indent, where the canteen sees them and where they can be cancelled, and each carries a mark unique to the case in its Remarks — both how the case finds its own indent afterwards and how whoever clears them down can tell them from a nurse's. Retries are off for that describe: a retry would file a second indent, and DI-05, DI-13 and DI-14 are counting indents.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DI-01 <code>diet-indent.regression.spec.ts:47</code>	Diet Indent > Add Indent opens from the Hospital module menu	Signed in; on the launcher.	1. Open the Hospital module and click Nurse Station. 2. Assert the page lands on Medicine Indent, then open Diet Indent. 3. Read the form, both dropdowns and the item table's headers.	Three priorities and the outlets from <code>DietIndentPage</code> ; columns from <code>ITEM_COLUMNS</code>	- The page opens on Medicine Indent — Diet Indent is a click away, which is the step this case proves. - Outlet, the pattern lookup, Remarks and the item table are all on screen. - Priority offers <i>Select Item</i> plus three entries — not the Medicine Indent list of six, which is the sharpest evidence on the page that the two tabs are separate forms. - Outlet offers <i>Select Canteen</i> plus its canteens; the item table's fourth column is unlabelled and holds the delete button.	P1	PASS 8.1 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DI-02 <code>diet-indent.regression.spec.ts:89</code>	Selecting a ward patient fills the indent form with that patient	Signed in; Add Indent open; a patient on the ward.	1. Assert the four patient boxes are empty and disabled before anything is clicked. 2. Click the ward card at the run's ward offset. 3. Read the patient panel back against the card.	Ward patient at the run's rotating ward offset; nothing is indented	• The boxes are disabled, so the ward card is the only thing that can fill them. • <i>Indent for</i> carries the patient's UHID, not the admission number the card was found by. • Name and admission date arrive; the form prints the bed as P(HCU-27) where the card prints HCU-27, so the card's bed is contained in the form's.	P1	PASS 4.8 s
DI-03 <code>diet-indent.regression.spec.ts:116</code>	Save without a priority is refused	Signed in; Add Indent open; a patient on the ward.	1. Select a patient, choose an outlet, add one food pattern. 2. Leave Priority unset and press Save.	Everything but the priority, so the refusal can only be about the priority	• Refused with <code>MESSAGES.noPriority</code>. • Refused, not saved: the line and the patient are still on the form — a save empties it.	P1	PASS 15.6 s
DI-06 <code>diet-indent.regression.spec.ts:135</code>	A quantity of zero should not be added to the indent	Signed in; Add Indent open; a patient on the ward.	1. Select a patient. 2. Offer a food pattern at a quantity of 0, stepping over any pattern the ward already has on order so the verdict is about the quantity.	Quantity 0	• Expected: the line is refused, and the refusal names the quantity. • Actual: a quantity of 0 is taken like any other and the line is added reading 1. The quantity is not recorded at all — every line lands as 1 whatever is typed, and reaches Verify Indent as 1. Raised as BUG-017; DI-10 records the same thing as an annotation.	P1	KNOWN FAIL 16.1 s · BUG-017

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DI-07 <code>diet-indent.regression.spec.ts:168</code>	A negative quantity is not accepted	Signed in; Add Indent open; a patient on the ward; a pattern on the form.	1. Set the quantity box to -5 and read it back. 2. Clear it and <i>type</i> -5 keystroke by keystroke; read it back.	Set and typed, because they are different paths into the control: fill writes the value, pressSequentially sends the keystrokes a nurse would, and a box can refuse one and take the other	• Neither path leaves a negative value in the box. • No minus sign survives typing.	P2	PASS 5.5 s
DI-08 <code>diet-indent.regression.spec.ts:198</code>	Save with the required fields empty is refused, one reason at a time	Signed in; Add Indent open.	1. Press Save on a form with nothing on it. 2. Add a patient and a priority, still nothing to send up; press Save again. 3. Press Enter on Quantity with no pattern picked.	Expected texts <code>MESSAGES.noItems</code> and <code>MESSAGES.noPattern</code>	• The empty form is refused for the indent table, and so is the form with a patient but no food pattern. • A line with no pattern is refused <i>at the line</i>, before Save is ever reached — Enter on Quantity is the add action. • Nothing reaches the item table.	P1	PASS 15.5 s
DI-08b <code>diet-indent.regression.spec.ts:225</code>	Save with an indent but no patient is refused	Signed in; Add Indent open.	1. Choose a priority and an outlet and add one food pattern — no patient. 2. Press Save.	Expected text <code>MESSAGES.noPatient</code>	• Refused, and the app names the patient precisely — the patient panel is the one required field a nurse cannot type into, so leaving it empty is the easiest mistake to make on this form. • The line stays on the form.	P1	PASS 10.4 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DI-10 diet-indent.regression.spec.ts:240	Several food patterns can be listed on one indent	Signed in; Add Indent open; a patient on the ward.	1. Select a patient and a priority. 2. Add three lines, each its own food pattern with its own remarks. 3. Read the item table back.	Three lines from <code>dietIndent(3)</code> , each with remarks <code><mark> line n</code>	• Three lines, three different patterns, in the order they were added, each carrying its own remarks. • <i>Deliberately not asserted:</i> that the quantities are the ones typed. They are not — the table reads 1 on every line. Recorded as an annotation so the report carries the evidence; DI-06 is the case that asserts it.	P2	PASS 28.0 s
DI-11 diet-indent.regression.spec.ts:276	Letters and symbols are not accepted as a quantity	Signed in; Add Indent open; a patient on the ward; a pattern on the form.	1. Type abc, then !@#, then 1a2 into the quantity box, keystroke by keystroke. 2. Read the box back after each.	Three rubbish values	• The box holds digits and nothing else, or nothing at all — anything it took from abc or !@# would show up as a character the pattern does not allow.	P2	PASS 6.8 s
DI-04 diet-indent.regression.spec.ts:339	A diet indent saves against an admitted patient WRITES	Signed in; a patient on the ward. Test timeout raised to 240 s; retries off.	1. Build a one-line indent: patient, outlet, priority, one food pattern. 2. Press Save.	One line from <code>dietIndent()</code> , remarks carrying the case's own mark	• Save answers <code>MESSAGES.saved</code>. • The form empties — the message alone is not proof, since the snackbar carries refusals too, in the same element.	P1	PASS 10.3 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DI-05 <code>diet-indent.regression.spec.ts:354</code>	Clicking Save twice files the indent once WRITES	Signed in; a patient on the ward. Test timeout raised to 300 s; retries off.	1. File a one-line indent and Save. 2. Press Save twice more with nothing touched in between — the double-click a nurse makes when the host is slow. 3. Find the indent on Verify Indent by its mark and count how many carry it.	One line; the case's own Remarks mark	• The first Save is acknowledged; the second and third are refused for an empty indent table. • Exactly one indent carries the mark. The form emptying is a weaker guard than a disabled button — it depends on the clear landing before the second click does — so the count on Verify Indent is what this case stands on.	P1	PASS 29.4 s
DI-09 <code>diet-indent.regression.spec.ts:378</code>	The remarks and quantity entered reach the saved indent WRITES	Signed in; a patient on the ward. Test timeout raised to 300 s; retries off.	1. File a one-line indent and Save. 2. Find it on Verify Indent by its mark and read the row.	One line; remarks are the only field a case can set and read back exactly	• Remarks survive the save intact. • The saved indent is for the food pattern that was ordered. • The quantity on Verify Indent agrees with the indent table — which is the same 1 for every line. Checked for consistency between the two screens, not against what was typed; DI-06 is the case about that.	P2	PASS 20.3 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DI-12 <code>diet-indent.regression.spec.ts:403</code>	A saved indent appears on Verify Indent as Pending WRITES	Signed in; a patient on the ward. Test timeout raised to 300 s; retries off.	1. File a one-line indent and Save. 2. Find it on Verify Indent by its mark and read the whole row.	One line; outlet and priority set from <code>dietIndent()</code>	- The indent carries a number of 10+ digits, the priority and the outlet it was filed under. - The grid prints the bare bed where the form printed P(Ex-HDU-02). - Status is Pending — waiting for the canteen — and it is filed under a user.	P1	PASS 14.6 s
DI-13 <code>diet-indent.regression.spec.ts:422</code>	A saved indent survives a reload, and is not filed twice by it WRITES	Signed in; a patient on the ward. Test timeout raised to 300 s; retries off.	1. File a one-line indent and Save; note its number. 2. Reload the browser — the nurse who is not sure the save took. 3. Read the form, then count indents carrying the mark.	A full reload, not a tab switch — the risk it covers is a form that re-posts what it still holds	- The reloaded form holds no line and no patient. - Exactly one indent carries the mark, under the same number as before, still Pending.	P1	PASS 24.5 s
DI-14 <code>diet-indent.regression.spec.ts:447</code>	Re-saving the same patient indent does not file it again WRITES	Signed in; a patient on the ward. Test timeout raised to 360 s; retries off.	1. File a one-line indent for a patient and Save. 2. Go back to the form, pick the <i>same</i> patient and offer the <i>same</i> food pattern. 3. Press Save with nothing in the table. 4. Count indents carrying the mark.	Expected text <code>MESSAGES.duplicatePattern</code>	- The repeat line is turned away <i>at Enter</i>, before Save is ever reached — that patient already has that food pattern on order. - With nothing in the table, Save has nothing to file either. - Still exactly one indent, under the original number.	P1	PASS 36.6 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DI-15 diet-indent.regression.spec.ts:484	Two patients get an indent each WRITES	Signed in; two different patients on the ward. Test timeout raised to 420 s; retries off. Files two indents.	1. Read two different ward patients. 2. File and save an indent for each, back to back, without reloading between them. 3. Find both on Verify Indent by their marks.	Two independent dietIndent() sets, each with its own mark	• Both saves are acknowledged and the form clears between them — one nurse can work down the ward without the form carrying anything over. • Two indents, two different numbers, two beds — not one indent that the second save overwrote.	P1	PASS 44.6 s

19. Regression — Diet Dashboard

Spec: tests/regression/diet-dashboard.regression.spec.ts · signed in. Route /hospital/nurse-station, the **Diet Dashboard** tab — where the ward sees every diet indent raised in a date range and starts or stops the meal a patient is actually on. This suite reads what section 18 writes.

What the two grids are, since nothing on the page says. They carry identical headers, no caption and no heading; only the button in the last cell differs. The upper grid is the diet each patient is currently on — one row per admission number, never more — and its button is **Stop**. The lower grid is everything else in the range, and its button is **Continue**. Status does not separate them: Pending, PR Submitted, Served and Reviewed rows turn up in both. DD-04 pins that down, and DD-12 is the case about it not being written anywhere on screen.

Run date note. Every case reads the live ward. A day the ward has not ordered on yet leaves the dashboard empty, which is the right answer and not a failure, so the cases that need rows skip rather than fail when there are none. In the 05 Oct run DD-03, DD-04 and DD-08 skipped for exactly that reason — downstream of **BUG-018**, which had the dashboard filtered to a day with nothing on it.

DD-14 ... DD-17 write. Each files its own diet indent through the Diet Indent tab first and drives Stop and Continue against that one, so the suite never stops a meal the ward ordered, and each leaves the ward as it found it. Retries are off for that describe: a retry would file a second indent, and these cases are counting rows.

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DD-01 <code>diet-dashboard.regression.spec.ts:110</code>	Diet Dashboard opens from the Nurse Station tab strip, with both grids	Signed in; on the launcher.	1. Open the Hospital module and click Nurse Station. 2. Assert the page lands on Medicine Indent — Diet Dashboard is two tabs along. 3. Open Diet Dashboard; read the filter bar and both grids' headers. 4. Read each grid's action buttons, where it has rows.	Nine columns from COLUMNS; buttons from ACTIONS	• Start Date, End Date, Search and Show are all on screen. • Both grids carry the same nine named columns plus a tenth for the button. • The upper grid offers only Stop and the lower only Continue. A quiet ward can leave either empty, so each is checked only when it has rows.	P1	PASS 10.3 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DD-02 diet-dashboard.regression.spec.ts:153	The dashboard opens filtered to today, and says so in the date boxes	Signed in; Diet Dashboard open.	1. Read the hidden Start Date and End Date inputs the app posts. 2. Read every row and check its Indent Date against today. 3. Read the date boxes the nurse actually sees.	Today, as <code>daysAgo(0)</code> builds it from the test host's local calendar	• Expected: the filter is today at both ends, and the boxes on screen show the range in force. • Actual, 05 Oct run — new: the hidden inputs came back 2026-10-06, a day ahead of the host. The dashboard opened on tomorrow, so the ward's own meals were not on it. Raised as BUG-018, and the reason DD-03, DD-04 and DD-08 skipped. • Also expected-to-fail: the boxes the nurse reads are empty regardless — placeholder text and nothing else — so the range in force is nowhere on screen. Raised as BUG-019.	P1	FAIL 46.5 s · BUG-018

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DD-03 <code>diet-dashboard.regression.spec.ts:192</code>	Every row carries a complete diet indent record	Signed in; Diet Dashboard open. Skips when the ward has not ordered a meal yet.	1. Read both grids. 2. Check every field of every row.	Statuses from STATUSES	- Every row carries an indent number of 10+ digits, an admission number of 8+, a patient, a bed and a diet. - Indent Date reads dd/MM/yyyy and Indent Time hh:mm AM/PM. - Status is one this suite knows. Remarks are not checked for content — they are the nurse's free text and are very often empty, which is not a defect.	P1	SKIPPED 7.1 s · no rows (BUG-018)
DD-04 <code>diet-dashboard.regression.spec.ts:224</code>	A patient is on one diet at a time, and no indent is in both grids	Signed in; Diet Dashboard open. Skips when no patient is on a diet today.	1. Read both grids. 2. Look for an admission number appearing twice in the running grid. 3. Look for an indent number in both grids at once.	Live ward data	- No patient is shown running two diets at the same time — this is the contract the upper grid keeps, and the reason the tab can offer a single Stop per patient. - No indent is in both lists; that would leave the nurse two contradictory buttons for one order. <i>Deliberately not asserted:</i> that status splits the two grids. It does not — the same status appears on both sides. Recorded as an annotation.	P1	SKIPPED 7.3 s · no rows (BUG-018)

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DD-05 <code>diet-dashboard.regression.spec.ts:260</code>	The date range filters both grids	Signed in; Diet Dashboard open. Test timeout raised to 180 s.	1. Filter to the last week and read both grids. 2. Check every row's Indent Date falls inside the window. 3. Filter to a window a month in the future.	window = last 6 days; future = 30–37 days ahead	• A week of a live ward brings back indents, and every one of them falls inside the window, in both grids. • A window no indent can fall in empties both grids rather than falling back to today.	P1	PASS 18.1 s
DD-06 <code>diet-dashboard.regression.spec.ts:295</code>	A date range that runs backwards is refused, not silently emptied	Signed in; Diet Dashboard open. Test timeout raised to 180 s.	1. Set the same week as DD-05 with its ends swapped — the range a nurse produces by setting the End Date first. 2. Assert both grids came back empty. 3. Look for any complaint on the page.	reversed = today ... 6 days ago	• Expected: a range the app cannot honour is said so, in the field or on screen. • Actual: both grids are emptied and nothing is said — which is also what an empty week looks like, and what a search with no matches looks like (DD-09). Nothing on the tab distinguishes them. Raised as BUG-020.	P2	KNOWN FAIL 26.4 s · BUG-020

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DD-07 diet-dashboard.regression.spec.ts:328	A wide date range is not silently cut off at 120 rows	Signed in; Diet Dashboard open. Skips if the ward raised fewer than 120 indents in the fortnight. Test timeout raised to 300 s.	1. Filter to the last fortnight; read both grids and check whether either is at the cap. 2. Filter to twice that window and read both again. 3. Compare the oldest indent in each grid across the two windows.	ROW_CAP = 120; windows of 14 and 28 days	• Expected: doubling the window brings back something older. A ward busy enough to fill a grid over a fortnight was busy over the fortnight before it too. • Actual: both windows come back with exactly 120 rows a grid and the same oldest indent in each. The list is cut at 120 and the rest is simply not there — no pager, no page-size box, no total anywhere on the tab. Five days of this ward is already enough to fill both grids. Raised as BUG-021.	P1	KNOWN FAIL 18.1 s · BUG-021

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DD-08 <code>diet-dashboard.regression.spec.ts:374</code>	Search filters both grids, on any column and in any case	Signed in; Diet Dashboard open. Skips when no diet indent was raised today to search for. Test timeout raised to 180 s.	1. Read both grids and take a seed row off whichever has one. 2. Search its indent number, then its patient name in two cases, then its bed. 3. Clear the box.	Seed row read live — which beds are occupied changes by the hour	• An indent number narrows the whole dashboard to that one row — the sharpest evidence that the box filters both grids at once. • A patient name matches in upper and lower case, and every row returned is that patient's. • A bed is a different column again, and matches. • Clearing the box puts the dashboard back the way it was.	P1	SKIPPED 7.1 s · no rows (BUG-018)
DD-09 <code>diet-dashboard.regression.spec.ts:427</code>	A search that matches nothing says so	Signed in; Diet Dashboard open.	1. Search a term no patient can match. 2. Assert both grids are empty. 3. Look for an empty-state message.	ZZ-N0-SUCH-PATIENT-ZZ	• Expected: an empty result says it is empty. • Actual: both grids are emptied to bare headers with nothing said — the same picture the tab shows for a quiet day and for the refused range in DD-06, so a nurse cannot tell a filter that matched nothing from one the app would not run. Raised as BUG-020.	P2	KNOWN FAIL 25.3 s · BUG-020

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DD-10 <code>diet-dashboard.regression.spec.ts:445</code>	Each grid sorts on its own, without disturbing the other	Signed in; Diet Dashboard open, filtered to the last week. Skips if either grid has fewer than two rows. Test timeout raised to 180 s.	1. Note the stopped grid's order. 2. Sort the running grid by patient name; read that column back. 3. Re-read the stopped grid's order. 4. Click the same header again.	Order is asserted as a <i>direction</i> rather than as the reverse of the first list: the grid cuts at 120 rows after sorting, so ascending hands back the first 120 names and descending the last 120	• The running grid goes into name order. • The stopped grid is exactly as it was — each grid carries its own sort. • A second click turns the same column round.	P2	PASS 20.3 s
DD-12b <code>diet-dashboard.regression.spec.ts:488</code>	The header says which column the grid is sorted by, and which way	Signed in; Diet Dashboard open, filtered to the last week. Skips if the running grid has fewer than two rows. Test timeout raised to 180 s.	1. Read the sort marks before anything is clicked. 2. Sort by Indent Date; read the marks on that column and on three others. 3. Click again and re-read. 4. Read the dates under the header.	Columns <code>indentNo,</code> <code>indentDate,</code> <code>patientName,</code> <code>status</code>	• Nothing is marked before a column is picked — the list arrives in the app's own order, which it does not claim to be any column's. • The caret goes on the sorted column alone, so it says <i>which</i> column the order is on and not merely that the grid is sorted. • It turns over with the order, and the mark agrees with the rows under it.	P2	PASS 19.6 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DD-11 <code>diet-dashboard.regression.spec.ts:542</code>	Indent Date and Indent Time sort chronologically, not as text	Signed in; Diet Dashboard open, filtered to the last week. Skips if the running grid has fewer than two rows. Test timeout raised to 180 s.	- Sort by Indent Date and parse the column as dates. - Sort by Indent Time and parse the column as minutes past midnight.	A week rather than today: a single day is all one date, and a ward's quiet hours are where AM and PM meet	- Indent Date is in date order — dd/MM/yyyy sorted as text would put the 1st of a month before the 28th of the one before it. - Indent Time is in time order — sorted as text, every PM would come before every AM of the same hour. - Which meridiems the rows covered is recorded as an annotation.	P2	PASS 20.2 s
DD-12 <code>diet-dashboard.regression.spec.ts:588</code>	Each grid is labelled, so a nurse can tell which list a row is in	Signed in; Diet Dashboard open.	1. For each grid, look for a caption, an <code>aria-label</code> or an <code>aria-labelledby</code> .	—	Expected: each grid is named, by a caption or an accessible label. Actual: the two are rendered identically — same classes, same nine headers, no caption, no heading above either card, no <code>aria-label</code> — and sit one above the other. Whether a patient is on the diet in front of you or was taken off it is readable only from the button at the end of the row. Raised as BUG-022.	P2	KNOWN FAIL 7.5 s · BUG-022

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DD-13 <code>diet-dashboard.regression.spec.ts:609</code>	Leaving the tab and coming back puts the dashboard back on today	Signed in; Diet Dashboard open. Test timeout raised to 180 s.	1. Widen the range to a week and type something into Search. 2. Go to the Diet Indent tab and back. 3. Read the search box, the hidden date inputs and every row.	Week window and a one-character search	• The tab refetches from scratch: the search box is empty again. • Expected: the range is back to today. • Actual, 05 Oct run — new: back to 2026-10-06, the same day-ahead default as DD-02. Raised as BUG-018. Worth pinning because the alternative — a stale list from the last visit — is what would make a nurse act on a diet that is no longer running.	P1	FAIL 1 m 14 s · BUG-018
DD-14 <code>diet-dashboard.regression.spec.ts:684</code>	An indent saved on Diet Indent appears as that patient's running diet WRITES	Signed in; a patient on the ward. Test timeout raised to 360 s; retries off. Files one indent.	1. File a diet indent through the Diet Indent tab and note its number. 2. Cross to the dashboard and search that number. 3. Read the row it lands in.	One line from <code>dietIndent()</code> , with the case's own Remarks mark	• The indent is in the running grid, against the right patient and the right bed. • It carries the food pattern ordered and the remarks typed. • Status is Pending and the action offered is Stop. • It is in one list, not both.	P1	PASS 21.6 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DD-15 <code>diet-dashboard.regression.spec.ts:721</code>	Stop takes the patient off the diet, and Continue puts them back WRITES	Signed in; a patient on the ward. Test timeout raised to 420 s; retries off. Files one indent, then acts on that one only.	1. File an indent and find it in the running grid. 2. Press Stop; read where the row went and what status it took. 3. Press Continue; read it back.	The case's own indent — a suite that reached for whatever row was at the top would be stopping meals a nurse ordered	• Stop moves the row to the stopped grid and rewrites the status to PR Submitted, leaving the diet itself alone; the running grid no longer shows it. • Continue closes the round trip exactly: same indent, same diet, same remarks, back in Pending and back in the running grid — which is also what puts the ward back as it was found. • <i>Recorded, not asserted:</i> neither button said anything. The row moving between two unlabelled grids is the entire feedback. DD-16 is the case about that.	P1	PASS 24.6 s

CASE ID	TITLE	PRE-CONDITIONS	TEST STEPS	TEST DATA	EXPECTED RESULT	PRI	STATUS
DD-16 <code>diet-dashboard.regression.spec.ts:769</code>	Stop asks before taking a patient off the diet they are on WRITES	Signed in; a patient on the ward. Test timeout raised to 420 s; retries off. Files one indent.	1. File an indent and find it in the running grid. 2. Press Stop, watching for a dialog and for a message from the moment the button is pressed. 3. Put the ward back with Continue — before reporting, because the assertion below fails on purpose and a failing test runs no further.	The case's own indent	• Expected: stopping a diet is clinical, and is asked about or at least acknowledged. • Actual: Stop acts on the first click. Nothing is asked and nothing is shown — by the time anything could be read, the row has moved and the status is written. The ward's own Diet Indent tab guards a far smaller thing, refusing a repeated food pattern outright (DI-14). Raised as BUG-023.	P1	KNOWN FAIL 23.8 s - BUG-023
DD-17 <code>diet-dashboard.regression.spec.ts:804</code>	Continue on an older indent warns that the running diet will be displaced WRITES	Signed in; a patient on the ward. Test timeout raised to 540 s; retries off. Files two indents for one patient.	1. File two indents for the same patient, so the ward has an order to go back to. 2. Confirm the newer is running and the older has been superseded. 3. Press Continue on the older one and read where the newer went. 4. Restore the ward: Continue the newer indent again.	Two independent <code>dietIndent()</code> sets against one admission	• The second indent supersedes the first on its own — that much the app does without being asked. • Continue on the older one drops the newer out of the running grid, correctly, since a patient is on one diet at a time (DD-04). • Expected: something says a live order was just cancelled to do it. Actual: nothing does — Continue is the quiet half of the same gap as DD-16. Raised as BUG-023.	P1	KNOWN FAIL 42.3 s - BUG-023

20. Known coverage gaps

GAP	WHY IT IS OPEN	SUGGESTED NEXT STEP
Grid search and row actions on the patient list (the old TC_REG_005–007)	Removed from the spec; the cases keyed off live data that was not stable enough to assert on.	Re-add against a patient the test registers itself, the way TC_FLOW_001 does.
Advance payment reaching the admission	Observed failing; see BUG-002. Asserting it today would make TC_FLOW_001 fail for a defect that is already tracked.	Add the assertion as soon as BUG-002 is fixed, so it guards the fix.
What is inside a printed admission document	TC_ADMD_007 checks that each Action button produces a real, non-empty PDF, not what the PDF says. Reading the text back would mean parsing it, and nothing in the suite does that today.	Add a PDF text extractor and assert the patient's name and admission number appear in each document.
The dashboard's list-level Print List / Print Package List, and the per-row cloud-arrow-down button	The two list buttons print the whole filtered page and are not tied to one admission. The cloud button produced nothing observable, and what it is meant to do is unknown.	Ask the module owners what the cloud button does, then cover all three.
Create-patient field validation (required fields, mobile/NID format, duplicate NID)	Every negative case would need the Confirm button, which writes to live data.	Needs a test tenant or a reset endpoint before it can be automated safely.
Firefox, WebKit and mobile viewports	Projects are present in the config but commented out — the host drops sessions under concurrent browsers.	Enable once the host tolerates more than one worker, or run them in separate scheduled passes.
Roles other than Administrator	Only one account is configured. Doctor, receptionist and accountant are declared as types but never exercised.	Add per-role accounts to .env and a storage state per role.
What the pharmacy and the canteen do next with an indent	The suite files indents and confirms they arrive as Pending. Partially Served, Served and Cancelled are read as tab labels (TC_NS_011) and never driven — moving an indent through them on a live ward would commit stock the hospital holds.	Cover the Verify Indent actions once there is a tenant that is not the hospital's own.
Consultancy services billed past the fifteen-row cap	CS-23 establishes the cap exists; nothing tests what a sixteenth charge does, because it cannot be read back or cancelled from the tab afterwards (BUG-016).	Cover it from the Accounts side, where the bill is presumably complete.
Diet Dashboard rows past 120, and the dashboard's own date boxes	DD-07 shows the grid cuts at 120 with no pager. The date boxes are read through the hidden inputs the app posts, because the visible flatpickr never mirrors them (BUG-019) — so no case asserts on what a nurse can actually see there.	Re-point the date assertions at the visible boxes once BUG-019 is fixed.
OPD, Pharmacy, Diagnostic, HR, Accounts	Out of scope for this cycle — only their tiles are checked on the launcher.	Plan per-module suites in the next cycle.

